PROJECT REPORT

Fake Review Detector (Pro Edition)

A Heuristic & Token-Density Based Chrome Extension

Project Domain	Browser Security, Natural Language Processing, Consumer Protection
Platform	Google Chrome Extension (Manifest V3)
Core Processing	Local, on-device heuristic analysis

Prepared as a technical and academic description of the supplied implementation context.

1. Abstract

Online consumer reviews influence purchasing decisions, yet e-commerce platforms are increasingly exposed to fabricated, incentivized, and automatically generated review content. The accessibility of large language models (LLMs) has lowered the cost of producing fluent, contextually plausible review spam, making simplistic keyword filters less dependable. This project presents **Fake Review Detector (Pro Edition)**, a browser-based review-scanning system implemented as a **Manifest V3 Chrome extension**. It discovers review content on supported and unfamiliar webpages, analyzes it locally using an 18-point heuristic engine, and communicates findings through contextual badges and a synchronized popup dashboard.

The system combines a three-tier DOM extraction strategy with a trust-score model and a token-density-based indicator of AI-like writing patterns. Rather than treating every stylistically polished review as wholly artificial, the engine distinguishes high-density AI fingerprints from lower-density patterns that may indicate AI-assisted editing. Results are presented as explainable signals, including reasons for score deductions, to help users assess reviews while browsing. Since the described engine is heuristic and does not rely on a trained classifier or external verification service, its outputs should be interpreted as risk indicators rather than definitive proof of deception or authorship.

2. Introduction

Customer reviews have become a central component of digital commerce. Consumers use star ratings, written experiences, and reviewer histories to estimate product quality and reduce uncertainty before making a purchase. This reliance creates an incentive for sellers and third parties to manipulate review ecosystems through fabricated testimonials, coordinated rating campaigns, and other forms of review abuse.

The recent spread of generative language models has changed the economics of this problem. LLMs can generate grammatically correct, varied, and persuasive text at scale. Compared with older spam that often contained conspicuous repetition or poor language quality, generated content may resemble ordinary consumer writing. Consequently, signals such as spelling errors or repeated phrases alone are insufficient for dependable screening.

The project addresses this challenge at the point where a consumer encounters reviews: the browser. The extension is designed to operate across e-commerce webpages without requiring each page to be sent to a remote analysis API. Local processing can reduce network latency and limit the transmission of review text to external services. Its objective is not to replace platform moderation or establish the factual truth of a review, but to provide an immediate, transparent layer of assistance.

Project objectives include:

- Discover review text and associated metadata across multiple website structures, including pages without a dedicated integration.
- Analyze review language using interpretable heuristics and a token-density measure.
- Separate strongly AI-patterned content from reviews that may have received limited AI-assisted editing.
- Display findings directly beside relevant review authors and synchronize aggregate results in the extension popup.
- Keep core analysis on-device and make deductions inspectable through explanatory feedback.

3. System Architecture

The extension follows the event-driven architecture of **Chrome Extensions Manifest V3**. Its principal components are the content script (**content.js**), the background service worker (**background.js**), and the popup interface. These components have distinct responsibilities and communicate through asynchronous extension messaging and Chrome storage.

3.1 Content Script

The content script executes in the context of eligible webpages and is responsible for discovering review elements, extracting text and related metadata, and inserting visual indicators. It observes page content and can respond to dynamically rendered reviews as the user scrolls. Extraction is separated from scoring so that website-specific DOM handling does not need to be duplicated inside the analysis engine.

3.2 Background Service Worker

The background component receives extracted review records through message passing and applies the heuristic engine. It computes the trust score, AI-pattern percentage, classification label, and explanatory reasons. Delegating analysis to the extension's background context avoids performing all processing in the page's main execution flow and helps maintain interface responsiveness. In Manifest V3, the background component is a service worker and is event-driven; it should not be assumed to remain continuously active.

3.3 Popup and Storage

The popup presents aggregate scan statistics and a list of flagged reviews. The described implementation uses **chrome.storage** as shared state and listens to **chrome.storage.onChanged** so that updates made during page scanning can be reflected in the popup. This separation allows the content script to focus on page interaction while the popup acts as a compact monitoring dashboard.

3.4 Processing Flow

- The content script discovers candidate review blocks and extracts review text and available author or rating information.
- Extracted records are sent asynchronously to the background service worker.
- The background engine applies the 18 heuristic checks, calculates a trust score and AI-token density, and returns the analysis result.
- The content script associates the result with the relevant review and inserts a badge, while the extension updates shared statistics and flagged-review records.
- The popup reads and observes stored state to display current totals and review-level findings.

The architecture is intended to keep the core analysis local. It does not require a third-party inference API for the described scoring path. Actual permissions, host access, and storage behavior are governed by the extension's manifest and implementation.

4. Methodology: Three-Tier DOM Extraction

E-commerce websites differ in their HTML structure, naming conventions, and rendering strategies. A single fixed selector is therefore unlikely to discover reviews reliably across all sites. The project uses a **Three-Tier Extraction Strategy**, applying increasingly general discovery methods.

4.1 Tier 1 — Site-Specific Profiles

The first tier uses targeted CSS selectors associated with known platforms, including Amazon, Myntra, and Walmart. Site profiles can identify review containers and their sub-elements, such as review body, author name, and rating. When a matching profile is available, it can provide higher precision and reduce the risk of treating unrelated page text as a review.

4.2 Tier 2 — Generic Selectors

If a site-specific profile does not produce usable results, the second tier searches for common web-development patterns such as classes or attributes containing terms like **.review-text**. This layer is intended to support websites whose markup follows common naming conventions even when no dedicated profile exists.

4.3 Tier 3 — Universal DOM Walker

The final tier uses a broader DOM traversal strategy. It searches the page for star-rating indicators and then walks upward through ancestor elements to locate a likely review container. Within the candidate region, it attempts to identify nearby review text and author information. This fallback increases potential coverage on unfamiliar sites, but its generality also creates a greater possibility of false matches—for example, when a rating icon belongs to a product summary rather than an individual review.

4.4 Extraction and Robustness Considerations

The tiers form a progressive fallback approach: specialized selectors are preferred, generic patterns are used next, and universal traversal is reserved for cases where structured extraction is unavailable. Extracted records should be normalized before analysis, with whitespace cleanup and safeguards against empty or duplicate text. As pages frequently load content asynchronously, discovery and injection must be designed to tolerate repeated scans and changing DOM nodes. The implementation's anti-duplication lock supports this requirement by preventing multiple badges from being inserted for the same review.

Universal discovery is a practical coverage mechanism, not a guarantee of compatibility with every website. Shadow DOM, iframe boundaries, virtualized lists, obfuscated markup, and frequent site redesigns can limit extraction. These constraints are appropriate targets for systematic testing and future refinement.

5. Algorithmic Approach: The 18-Point Heuristic Engine

The background engine begins each review with a **Trust Score of 100**. It then evaluates 18 checks grouped into two conceptual sections: human-spam indicators and AI-writing fingerprints. When a check is triggered, the engine applies a configured penalty and records a reason that can be surfaced to the user. The final score and label therefore depend on the particular checks and penalty weights encoded in the implementation.

The score is an interpretable heuristic index, not a calibrated probability that a review is genuine. A high score means fewer configured warning signals were detected; it does not independently verify a purchase, reviewer identity, or product experience. Similarly, a low score indicates multiple suspicious patterns rather than conclusive evidence of fraud.

5.1 Section A — Human-Spam Indicators

The first section targets conspicuous patterns commonly associated with low-effort or promotional spam. The supplied design includes:

- **Excessive capitalization:** all-caps text or shouting-style capitalization.
- **Punctuation anomalies:** repeated exclamation marks or other excessive punctuation.
- **Character repetition:** elongated forms such as “looooove,” which may indicate emphasis or spam-like writing.
- **Emoji density:** unusually frequent emoji use.
- **Extreme ratings without explanation:** very high or low ratings paired with little substantive reasoning.

These checks are lightweight and computationally inexpensive. However, they can also flag authentic expressive writing, short reviews, or language and cultural styles in which such patterns are common. For this reason, they are most useful as combined indicators rather than independent proof.

5.2 Section B — AI-Writing Fingerprints

The second section looks for stylistic and lexical patterns associated with generated or heavily edited review text. Examples supplied for the project include:

- **Punctuation style:** disproportionate use of em-dashes.
- **Formulaic openings:** constructions such as “I recently purchased…”.

- **Participial or purpose chains:** phrases such as “making it perfect for…”.
- **Characteristic vocabulary:** words such as “versatile” and “elegance” when detected by the engine's pattern rules.
- **Balanced hedging:** symmetrical constructions such as “not too tight, not too loose.”

These features are treated as fingerprints because they may occur more frequently in templated or LLM-assisted writing. They are not unique to AI. Human writers can use the same vocabulary and structures, and generated text can avoid them. The system consequently measures the presence and density of configured indicators instead of claiming to identify authorship with certainty.

5.3 Token-Density Algorithm

The token-density component converts detected AI-style patterns into a normalized indicator. Each matched fingerprint contributes one or more **AI Tokens** according to the implementation's matching rules. Let *A* represent the number of detected AI tokens and *W* the number of words in the review. The denominator uses a minimum floor of 30 words to reduce extreme percentages caused by very short texts.

AI Percentage = 100 × A / max(W, 30)

For example, if a review contains 12 counted AI tokens and 40 words, its indicator is 30%. If the same number of tokens occurs in a review with only 15 words, the floor makes the denominator 30, yielding 40% rather than 80%. This floor reduces short-text inflation, although it does not eliminate uncertainty for brief reviews.

The term “AI Percentage” should be understood as a **heuristic fingerprint-density score**, not a statistically calibrated probability that a language model wrote the review. Its numerical precision is arithmetic precision; predictive accuracy would require validation against a labeled dataset representing real reviews, human-written spam, AI-generated content, and AI-assisted writing.

5.4 AI Bifurcation: AI Spam versus AI-Assisted

A central design decision is to avoid treating all detected AI-style writing identically. The engine uses the computed percentage to distinguish between two broad cases:

- **High-density or 100% AI-patterned spam:** reviews whose detected fingerprint density reaches the engine's full-pattern threshold receive a heavier penalty and are presented as strongly suspicious.
- **AI-assisted or polished writing:** reviews with a lower density of AI fingerprints receive a lighter penalty and may be labeled “AI Assisted,” reflecting the possibility that a human-authored review was edited or polished using tools such as Grammarly or ChatGPT.

This bifurcation is intended to reduce the conflation of writing assistance with fabricated experience. It is nevertheless a rule-based distinction: the percentage cannot establish whether a real purchase occurred or whether a human supplied the underlying account. Thresholds and penalties should be documented in the source code and evaluated empirically before the labels are interpreted as reliable classifications.

5.5 Decision and Explainability

The engine combines the deductions and pattern findings into a final trust score and user-facing category, such as verified, suspicious, or AI-assisted, according to configured thresholds. Each triggered rule is retained as an explanation. This supports transparency: rather than displaying only a label, the extension can communicate the observed reasons, such as excessive punctuation or a detected formulaic phrase.

The term “Verified” in the interface should be interpreted as a product label for a review that passes the extension's configured checks, unless the implementation independently verifies purchase status through a trusted platform source. A heuristic browser extension alone cannot establish platform-level verified-purchase status.

6. User Interface Design

6.1 In-Page Badging

Analysis results are inserted into the webpage as compact, custom-styled pill badges adjacent to the review author's name. The described visual states include ✓ **Verified**, ■ **Suspicious**, and ■ **X% AI**. Placing the result in context reduces the need for users to switch between the shopping page and a separate analysis tool.

A strict anti-duplication lock prevents repeated scans from adding multiple badges to the same review. This is important on dynamic pages where content can be re-rendered or appended as the user scrolls. Badge insertion should avoid disrupting the original page's layout and should use distinctive styling that remains legible against varying site themes.

6.2 Explainable Tooltips

Hovering over a badge reveals a tooltip listing the reasons behind the score deductions or classification. This is an example of **explainable user-facing automation**: the system exposes observable patterns that contributed to its decision instead of presenting an unexplained verdict. The tooltip also helps users distinguish between human-spam signals and AI-style writing signals.

Explanations should be phrased cautiously—for example, “formulaic opening detected” rather than “written by AI”—because individual linguistic features are not conclusive evidence of authorship. Accessible behavior, including keyboard focus and readable contrast, would further improve usability.

6.3 Neumorphic Popup Dashboard

The popup follows a **Neumorphic (Soft UI)** visual language, using soft raised and inset shadows, restrained surfaces, and a custom SVG logo. Its dashboard summarizes total reviews scanned, verified-count results, and suspicious findings, and provides a list of flagged reviews. The design aims to communicate status and aggregate information within the limited dimensions of a browser-extension popup.

The popup listens for **chrome.storage.onChanged** events so that scan totals and flagged-review records can update as the user scrolls and the content script processes additional reviews. This event-driven synchronization avoids requiring users to close and reopen the popup to see every change. Clear labels, meaningful empty states, and restrained use of color are important to ensure that visual polish does not obscure the uncertainty of heuristic results.

7. Conclusion & Future Scope

Fake Review Detector (Pro Edition) demonstrates a practical browser-level approach to review screening by combining cross-site DOM discovery, local heuristic analysis, and contextual user-interface feedback. Its three-tier extraction strategy is designed to balance precision on known platforms with fallback coverage on unfamiliar pages. The 18-point engine combines traditional spam indicators with AI-style linguistic fingerprints, while token density and AI bifurcation provide a mechanism for expressing degrees of detected pattern concentration. In-page badges, explanatory tooltips, and a live popup dashboard make the results accessible during ordinary shopping.

The principal limitation is that linguistic heuristics cannot reliably establish whether a review is fake or AI-authored. Writing style varies across people, languages, products, and contexts; meanwhile, generated text can imitate human variation. The trust score and AI percentage should therefore be presented as screening signals and not as ground-truth probabilities. The system's claims of accuracy would require controlled evaluation.

7.1 Future Scope

- **Dataset-based evaluation:** build a labeled corpus containing authentic reviews, human-written spam, LLM-generated reviews, and AI-assisted reviews across multiple product categories and languages.
- **Metric reporting:** measure precision, recall, F1-score, false-positive rate, and calibration where applicable; report results separately by site, language, and review length.

- **Threshold optimization:** tune penalties and density thresholds on a training/validation split, with a held-out test set to avoid overfitting.
- **Multilingual support:** expand linguistic patterns and normalization for Indian and other languages, including code-mixed text.
- **Robust extraction:** improve handling of infinite scrolling, shadow DOM, iframe content where permitted, and frequently changing site markup.
- **Privacy and security review:** minimize stored review text, define retention behavior, and audit extension permissions and content-script boundaries.
- **Model-assisted optional mode:** explore an optional locally hosted or privacy-preserving classifier, while retaining transparent explanations and a fully local baseline.
- **Usability testing:** assess whether users understand the difference between suspicious, AI-assisted, and platform-verified labels, and whether badges affect purchasing decisions appropriately.

With empirical validation, careful calibration, and continued compatibility testing, the extension could evolve from a heuristic proof of concept into a more rigorously evaluated consumer-assistance tool. Its strongest design principle is the combination of local processing and transparent signals, while maintaining clear limits on what automated text analysis can prove.

Appendix A. Terminology

Term	Definition
DOM	Document Object Model; the structured representation of a webpage's elements.
Manifest V3	Chrome extension platform architecture using service workers for background event handling.
Heuristic	A rule-based indicator or practical decision method that is not necessarily statistically optimal.
Token density	The count of matched AI-style indicators normalized by a word-count denominator.
AI-assisted	Content that may have been edited or polished with AI tools; this label does not prove authorship.
Explainability	Presentation of understandable reasons or signals behind an automated result.